

Finance Workflow Management Tool – Feature Set (Agile Backlog)

Project Overview

This project is a **B2B Finance Workflow Management Tool** designed to streamline common financial and operational approval processes. It combines contract renewal tracking, approval workflows, and proposal-to-invoice generation into one integrated hub. The goal is to help businesses “**automate and track approval workflows**” such as contract renewals, expense approvals, purchase orders (PO), and sales proposals. Users can submit requests through forms, managers and finance teams can approve them in sequence, and the system tracks statuses and sends automatic reminders. This ensures nothing falls through the cracks – contracts are renewed on time, expenses get approved efficiently, and sales proposals convert to invoices without manual hassle. The product will provide a basic dashboard for status tracking and timely notifications to relevant stakeholders, improving transparency and efficiency across these processes.

Key user roles and benefits:

- *Employees (Requestors)*: submit requests (e.g. contract renewal, expense report) and track their status in real-time.
- *Managers (Approvers)*: get notified of pending approvals and can approve/reject requests with a click, speeding up decision cycles.
- *Finance/Administrators*: review approved requests (if needed) for final sign-off, manage contracts and subscriptions, and ensure financial compliance.
- *Organization Executives*: view dashboard reports of ongoing requests, upcoming contract expirations, and overall process metrics for better oversight.

By integrating multiple workflows into one tool, the system reduces email back-and-forth, provides a **central source of truth** for contract and request information, and automates routine follow-ups (like renewal reminders and invoice due notices). This not only saves time but also helps avoid costly lapses (e.g. forgetting to renew a critical software license or delaying an invoice). The project is currently in early planning, and an Agile Scrum approach will be used for development – delivering incremental value through defined sprints and continually incorporating user feedback.

Technology Stack & Platform

The solution will be built on a modern cloud stack to ensure scalability and real-time responsiveness:

- **Firebase Platform**: Utilizing **Firestore** as the cloud NoSQL database for storing all data (requests, contracts, proposals, etc.) and **Firebase Cloud Functions** to handle backend logic and automation. This serverless architecture provides real-time data sync, so any updates (like an approval action or a new reminder) reflect instantly in the app. It also simplifies implementing triggers (e.g. to send an email when a request is submitted or to schedule reminder notifications).

- **Web Application:** Developed in **React** with **TypeScript**, providing a robust, type-safe front-end. React will allow creating a responsive and interactive UI for the dashboard, forms, and workflow tracking. Users (with proper authentication) can access the web app from any browser.
- **Authentication & Security:** Likely using **Firebase Auth** for secure login and user account management. Role-based access control will ensure that users only see and do what their role permits (e.g. only managers see the "Approve" button, only finance sees finance-specific data). Data will be stored securely, and all communications are over HTTPS.
- **Mobile App (Future):** A **React Native** mobile app is planned for future development, reusing much of the web app's logic and UI design for consistency. This will allow on-the-go access to approve requests or check statuses, with push notifications for reminders.
- **Integrations (Future Considerations):** The architecture will allow integrating email and possibly chat (Slack/Teams) for notifications. For the proposal/invoice module, integration with payment gateways (e.g. Stripe or PayPal) is anticipated to enable payment links. The Firebase backend can also integrate with other services via APIs if needed (for example, connecting to an ERP or CRM in later phases).

Development will follow Agile practices: features are organized into epics and user stories (detailed below), and we will implement them in short sprints. Continuous testing and code reviews will ensure quality, and the Firebase backend will let us deploy updates without downtime. This setup supports rapid iteration – we can quickly add or refine features based on user feedback in true Scrum fashion.

Product Backlog (Epics & User Stories)

Below is an organized feature set broken into **major epics** (high-level feature groups) and their constituent **user stories**. Each user story is written from an end-user perspective, capturing the desired functionality and benefit. This backlog will guide development; highest priority (MVP) items will be tackled in the first sprints, while others will follow in subsequent iterations.

Epic 1: Contract Renewal & License Tracker

Description: This epic covers the management of company contracts and software licenses, ensuring they are renewed on time through an approval workflow. It allows users to submit renewal requests which pass through managerial and financial approval. The system keeps a record of all contracts/licenses, their terms, and expiration dates, sending proactive alerts to prevent any lapse. This reduces the risk of forgotten renewals and keeps subscriptions up-to-date.

- **User Story:** *As an employee or department head, I want to submit a contract or license renewal request (with key details like vendor, cost, renewal date) so that the approval process for renewing that contract is initiated.*
Acceptance Criteria: A request form is available where the user enters contract details and justification. Upon submission, the request is recorded in the system with status "Pending Manager Approval." The requester can view their request in the dashboard.
- **User Story:** *As a manager, I want to review a submitted contract renewal request from my team and approve or reject it, possibly with comments, so that I can control which renewals proceed.*
Acceptance Criteria: Managers receive a notification when a new request awaits their approval. They can open the request details, see all provided info, and either **Approve** or **Reject**. If rejected, the system records the reason and notifies the requester. If approved, the request's status updates to "Approved by Manager" and it moves to Finance review.
- **User Story:** *As a finance team member (or budget owner), I want to review manager-approved renewal requests and give a final sign-off, so that I ensure the renewal fits the budget and compliance before execution.*

Acceptance Criteria: Once a manager approves a request, a finance user can mark it as **Final Approved** (or reject if issues are found). Upon final approval, the system records that the contract is cleared for renewal. Optionally, this could trigger the actual renewal process (e.g. notify procurement to renew or even auto-renew if integrated with vendor). The request status becomes "Completed" and the requester is notified that the renewal will be processed.

- **User Story:** *As an IT or finance admin, I want the system to record all active contracts and licenses in a central repository, so that I can easily access contract details, terms, and expiration dates in one place.* ¹

Acceptance Criteria: There is a module/page listing all contracts and licenses with key information (vendor, start date, end date, value, owner, etc.). Users with permission can add or update contract entries. All documents or attachments (like the contract file) can be stored securely and linked to each record. This **centralized contract repository** makes it easy to find contract details when needed, eliminating the need to search through emails or files.

- **User Story:** *As the system (automated function), I want to send timely alerts and reminders to relevant users before a contract's expiration or renewal due date, so that no renewal deadline is missed.* ²

Acceptance Criteria: For each contract in the system, a reminder email (and in-app notification) is sent, for example **30 days** and **7 days** before the expiration date. The manager/owner of the contract and/or a finance admin receives the alert. The reminder message includes contract details and prompts the user to take action (submit renewal request or review options). If a renewal request is already submitted, the reminder may be suppressed or adjusted. This **automated reminder** feature ensures proactive management of renewals, a key benefit of the system (no more missed deadlines).

- **User Story:** *As a procurement or finance user, I want to compare renewal options (such as different plans or vendors) for an upcoming contract renewal, so that I can choose the most cost-effective or suitable option.*

Acceptance Criteria: When a contract is nearing renewal, the system allows recording multiple renewal proposals (e.g., continue current plan vs. upgrade to a new plan, or quotes from different vendors). The user can input alternative costs or terms. These options can be attached to the renewal request for evaluation. Managers/finance can see a side-by-side comparison in the approval interface and make a decision. (This could be a later enhancement – providing a structured way to evaluate and maybe even recommend alternatives.)

- **User Story:** *As a user in any role, I want to view the status and history of contract renewal requests, so that I know where each request stands and can audit past renewals.*

Acceptance Criteria: The system maintains a log of each request's progression (submitted → manager approved on date → finance approved on date → completed). A user can click on a contract or request to see its timeline and any comments. This provides transparency and helps in auditing or reviewing what happened for each renewal.

Epic 2: Approval Workflow Hub (Expenses & Purchase Orders)

Description: This epic represents the **general workflow engine** of the tool which can handle multiple types of approval processes, not just contract renewals. Initially, the focus will be on common financial workflows like **Expense Reimbursements** and **Purchase Order (PO) Approvals**, as these are typical pain points in organizations. The system will provide forms for these request types and route them through the appropriate approval chain (e.g. employee submits expense → manager approves → finance/payroll processes it). A **unified dashboard** will show all requests (of any type) and their

statuses, acting as a one-stop hub for all pending approvals. In the future, this workflow capability can be extended or customized to new processes as needed, making the system very flexible ³.

- **User Story:** *As an employee, I want to submit an expense reimbursement request with details (amount, category, receipts) so that I can get approval and reimbursement for out-of-pocket expenses.*
Acceptance Criteria: A dedicated form exists for expense requests. It captures date, amount, expense category (travel, office supplies, etc.), and allows attachment of receipt images. On submission, the request goes to the employee's manager for approval. The employee can track this in the dashboard with status "Pending Manager Approval."
- **User Story:** *As an employee, I want to submit a Purchase Order request for a new purchase or vendor payment, so that it can be approved before I place the order.*
Acceptance Criteria: There is a PO request form capturing vendor info, item/service to purchase, cost, and justification. On submission, it routes to the appropriate manager or designated approver. The status is tracked similarly. (The system might allow different routing logic here, e.g. POs over a certain amount go to a senior manager or procurement department – such rules can be configured by an admin.)
- **User Story:** *As a manager, I want to approve or reject expense and PO requests submitted by my team, possibly adding comments, so that I can control spend and ensure policy compliance.*
Acceptance Criteria: Managers have a centralized "Approvals" queue where they can see all pending requests (contracts, expenses, POs, etc. filtered by type). Each request shows relevant details (for an expense: who, how much, why, receipt; for a PO: vendor, amount, purpose). The manager can approve or deny each, with an optional comment. On approval, the next step is triggered (e.g. finance review or auto-completion if none required). On rejection, the requester is notified with the comment. Managers should find the interface simple: e.g. a list of pending items and an **Approve/Reject** button on each or in detail view.
- **User Story:** *As a finance team member, I want to review approved expense/PO requests when needed (especially larger amounts or certain categories), so that I can do a final compliance check and then mark them for processing.*
Acceptance Criteria: The workflow can be configured such that some requests require finance approval after manager approval (for example, any PO over \ \$10k might need finance approval). The system will place those in a Finance queue after managerial approval. Finance users can view details and then mark as approved (or send back if issues). Once finance approves, the request is completed and tagged for payment processing externally. If a request doesn't need finance review, a manager's approval can auto-complete it. This configuration can be adjusted per workflow type by an admin.
- **User Story:** *As an admin, I want to be able to configure different approval workflows (steps and approvers) for each request type, so that the system can support our organization's unique processes and rules.* ³
Acceptance Criteria: There is an administrative interface (or configuration file in early version) where we can define workflow templates. For each type of request (contract renewal, expense, PO, etc.), define the sequence of approval: e.g. *Expense under \ \$1k: Manager only; Expense over \ \$1k: Manager then Finance; PO under \ \$5k: Manager → auto-complete; PO over \ \$5k: Manager → Director → Finance*, etc. The system uses these rules to route requests automatically to the right approvers. This **customizable workflow** capability ensures the tool can adapt to different company policies (e.g., multi-level approvals, auto-approvals under thresholds, etc.) without hard-coding each scenario.
- **User Story:** *As any user (requestor, manager, finance), I want a dashboard that shows a summary of all my pending tasks and recent requests, so that I can quickly grasp what needs attention and what's the status of various items.*
Acceptance Criteria: The application's main screen is a dashboard. For an approver (manager/finance), it lists pending approvals assigned to them (e.g. "3 expense reports pending your

approval, 1 contract renewal pending review"). For a requester, it shows their submitted requests and their current statuses ("Expense X – approved by manager, pending finance"; "Contract Y – completed on [date]"). It may also highlight upcoming contract expirations or tasks. The dashboard should be filterable or segmented by request type and have basic metrics (e.g., total approved this month, etc.). This provides a one-glance view of the workflow hub's activities.

- **User Story:** *As a system, I want to send automatic notifications and reminders for approval tasks, so that approvals happen in a timely manner and nothing is left waiting unnecessarily.*

Acceptance Criteria: When a user submits a request, the designated approver receives an immediate email (and/or app notification) about the new request. If a request sits for, say, 3 days without action, the system sends a reminder to the approver (and possibly CC a higher-up if escalation is needed). Similar reminders occur for finance review if delayed. Also, daily or weekly summary emails could be sent ("You have 2 pending approvals"). This ensures approvers are nudged to take action, which speeds up the overall process.

- **User Story:** *As an admin or analyst, I want to generate basic reports on the workflow (e.g. how many requests of each type, average approval time, etc.), so that I can identify bottlenecks or high spending areas.*

Acceptance Criteria: The system can export or display reports such as: number of expense reports submitted vs approved in a period, average time taken at each approval stage, list of all contracts up for renewal in next quarter, etc. (This may be a later enhancement, but even in MVP, having at least an export of data or simple stats on the dashboard adds value for process improvement.)

Epic 3: Integrated Proposal & Invoice Generator

Description: This epic focuses on the sales finance side – creating proposals (quotations) for clients and seamlessly turning them into invoices once approved. The tool will let users build proposals using templates, send them for approval (internally or to the client), and upon approval, automatically generate an invoice record with the same details. The status of each proposal/invoice is tracked (e.g. Draft → Sent → Accepted → Invoiced → Paid), providing visibility into the sales pipeline and receivables. This integration eliminates the need to re-enter data in a separate invoicing system, reducing errors and speeding up the quote-to-cash process ⁴. (Note: This module may be introduced after the core internal workflows are in place, unless it's a key part of the MVP.)

- **User Story:** *As a sales or finance user, I want to create a sales proposal for a client by filling in a template with our products/services and pricing, so that I can quickly generate a professional quote to send for approval.*

Acceptance Criteria: The system provides proposal templates (which can be customized with company branding). The user can input client details, line items (description, quantity, price), discounts, and terms. The proposal calculates totals and can be previewed as a nicely formatted document (PDF or web). The proposal is saved with status "Draft" or "Pending Approval". (If internal approval is needed before sending to client, it could route to a manager's approval similar to internal workflows, or the user can directly send to client if they have that authority.)

- **User Story:** *As a sales manager (or designated approver), I want to review and approve a proposal before it's sent to the client, so that I ensure pricing and terms are correct and acceptable.*

Acceptance Criteria: If the organization's process requires it, proposals appear in an internal approval queue (just like other requests) for review. The manager can approve it, which then allows the sales user to send it out to the client. If changes are needed, the manager can reject with comments (the proposal returns to Draft for editing). This adds a control step for high-value deals or special pricing. (For a simpler flow or smaller companies, this step could be optional.)

- **User Story:** *As a client (external to the system), I want to receive a proposal and indicate approval (acceptance) or rejection electronically, so that the selling company knows my decision quickly.*

Acceptance Criteria: Once a proposal is approved internally and ready, the sales user can send it to the client via a shareable link or email. The client opens a web view of the proposal (no login required, maybe a secure tokenized link). They can see a summary and download the PDF. The client has options to **Accept** or **Decline** the proposal (and possibly add a note or signature if needed). When they accept, the system records the proposal as “Accepted” (with timestamp). If declined, it’s marked accordingly and feedback is captured. The sales team is notified of the response.

- **User Story:** *As a system, I want to automatically convert an accepted proposal into an invoice record, carrying over all pertinent details, so that the finance team can quickly proceed to billing without re-entering data.* ⁴

Acceptance Criteria: When a proposal’s status changes to “Accepted” by the client, the system generates a corresponding **Invoice** entry. The invoice will include the same line items, pricing, client info, etc., essentially a copy of the proposal details but marked as an invoice. This should happen with one click or automatically upon acceptance – “Turn approved proposals into invoices with a single click—no re-typing” ⁴. The invoice is initially marked “Unsent” or “Draft”. The finance user can review it and then mark it “Sent” (to the client) when they actually send the official invoice (or the system could even email it out). This integration ensures a seamless flow from proposal to billing, reducing errors and effort ⁵.

- **User Story:** *As a client, I want to pay an invoice online via a convenient link, so that I can complete the transaction quickly and securely.*

Acceptance Criteria: When the invoice is sent to the client (likely via email with a PDF/link), it includes a **payment link** (if online payments are enabled). Clicking the link leads to a secure payment page (could be powered by Stripe/PayPal etc.) where the client can pay by credit card or other methods. Once payment is made, the system updates the invoice status to “Paid” and notifies the finance team and the sales person. (If online payment isn’t integrated in MVP, at least the system can record if the client marked the invoice as paid or if finance manually marks it paid when money is received.)

- **User Story:** *As a finance/accounts receivable user, I want to track the status of proposals and invoices in the system, so that I have visibility into outstanding proposals and unpaid invoices at any time.*

Acceptance Criteria: The dashboard or a dedicated section shows all proposals and their statuses (e.g., 5 proposals sent, 3 accepted, 2 awaiting client), and all invoices (unpaid vs paid, with amounts). It should be easy to filter by client or date. Overdue invoices (if any) could be highlighted. There could also be automated reminders for unpaid invoices (e.g., email the client if X days past due). This helps the finance team stay on top of receivables without a separate tracking spreadsheet.

- **User Story:** *As an admin, I want to manage proposal templates and settings, so that our proposals and invoices meet our branding and business needs.*

Acceptance Criteria: The system allows uploading or editing templates (logos, company info, standard terms and conditions). It also lets setting default tax rates, currencies, and other parameters so that proposals/invoices are consistent. Admin can also configure if internal approval is required for proposals above certain values, etc. These settings ensure the module is flexible for different companies.

Epic 4: User Management & Security

Description: This epic includes features related to managing user accounts, roles, and permissions within the system – crucial for a B2B application where multiple employees and departments will use the tool. It ensures the right people have access to the right functions (for example, only managers can

approve requests, only finance can see finance sections, etc.). This also covers authentication and basic security auditing.

- **User Story:** *As an admin, I want to invite or register users into the system and assign them roles (employee, manager, finance, admin), so that each person has appropriate access.*
Acceptance Criteria: There is a user management interface where an admin can add new users (or integrate with the company SSO in future). When adding a user, the admin sets their role. Roles define permissions: e.g., **Employee** can submit requests and view their own; **Manager** can do that plus approve for their team; **Finance** can access all financial review screens; **Admin** can configure system settings and manage all data. Users receive an invite email to set their password (if not using SSO).
- **User Story:** *As an admin, I want to define the manager for each employee (or an org chart hierarchy), so that approval workflows can automatically route to the correct manager.*
Acceptance Criteria: In the user management, the admin can specify who each user's direct manager is (or upload an org structure). Alternatively, integrate with an HR system if available. The system uses this info to send requests to the right approver. If a user's manager changes, updating it here affects future requests. There might also be an option to designate backup approvers or a chain of escalation if someone is out of office.
- **User Story:** *As a user, I want to authenticate securely (log in) to the web app (and mobile app, when available) via email/password or SSO, so that I can access the system in a secure manner.*
Acceptance Criteria: The login process is protected (using Firebase Auth or similar). Passwords are never stored in plain text, and appropriate password policies or SSO integration are in place. Users can reset their password via email. Sessions time out after a period for security. On mobile, biometric login could be enabled in the future.
- **User Story:** *As a security auditor or admin, I want the system to keep audit logs of key actions (login, submissions, approvals, changes), so that we have traceability for compliance and troubleshooting.*
Acceptance Criteria: Every important action generates a log entry: e.g., when a user submits a request, when a manager approves or rejects (including their comment), when a finance person marks as done, when a proposal is sent/accepted, and when any configuration is changed. These logs can be viewed by admins and are stored in a secure manner. This helps in case of disputes ("Who approved this and when?") and for compliance if the processes are audited.

Epic 5: Dashboard & Notifications (Cross-Cutting Features)

Description: This epic highlights the **user interface and notification system** that is shared across the platform. It ensures that all the information is presented clearly to users and that they are kept informed in real-time of any items requiring their attention. (Some of these were touched on in earlier epics, but here we consolidate cross-cutting aspects.)

- **User Story:** *As a user, I want a unified dashboard when I log in that gives me a snapshot of all relevant information (approvals, requests, tasks, reminders), so that I don't have to navigate to multiple screens to know what's going on.*
Acceptance Criteria: The dashboard is the home screen after login. It shows sections or cards for: **Pending Approvals For You** (if you're a manager/approver), **Your Recent Requests** (with statuses), **Upcoming Contract Expirations** (if any in next 60 days), and maybe **Summary Stats** (like number of requests completed this month). Each item is a link to the detailed page for that request or contract. The UI is clean and not cluttered, focusing on actionable items. Users can customize or filter what they see to some extent.
- **User Story:** *As a user, I want to receive notifications (email and in-app) for important events (like when my request is approved or when I have a new task), so that I stay informed even when not actively watching the dashboard.*

Acceptance Criteria: The system sends an email when: an approval is needed from a user, when a user's own request gets approved or rejected, when a proposal is accepted by a client, when an invoice is paid, etc. In-app notifications (a bell icon or similar) also show these alerts for those actively using the app. Notification content is concise and provides a link to view details. There is an option for users to configure their notification preferences (e.g., turn off email for certain events if they only want in-app, etc.).

- **User Story:** *As a manager or exec, I want the dashboard to provide some analytics or reports (like charts of requests over time, or pie chart of approved vs rejected), so that I can get insights at a glance.*

Acceptance Criteria: In addition to lists of items, the dashboard can have a few visual widgets: e.g., a bar chart of number of requests per month, or a breakdown of request types in a period. Perhaps a chart of total spending approved in expenses vs budget. These help identify trends (like increase in software contract spend). This might be a stretch goal for MVP, but at least a simple metric display (like "25 requests completed this quarter") is included to add value.

- **User Story (Future):** *As the system, I want to send scheduled summary reports to relevant people, so that managers or admins can get a periodic email of activity without logging in.*

Acceptance Criteria: (Future enhancement) The system can compile a weekly summary email, e.g. every Monday morning: "Past Week: 10 new requests submitted, 9 completed. 2 Contracts expiring next month." This goes to interested users (maybe admins or department heads). This keeps stakeholders in the loop and demonstrates the tool's usage and value automatically.

MVP Scope and Future Enhancements

In line with Agile methodology, we will first build a **Minimum Viable Product (MVP)** focusing on core functionalities, then iterate with additional features. Below is a breakdown of what will likely be in the MVP versus what can be planned for later sprints:

- **MVP Focus:**

- Basic workflow for at least **two request types**: for example, *Expense Reimbursement* and *Contract Renewal*. These will prove out the request submission, approval routing (manager → finance), and notifications.
- **Contract tracking** with ability to store contracts and get renewal reminders (basic version).
- A simple **dashboard listing** of pending approvals and submitted requests.
- **Proposal to Invoice conversion** basic flow: the ability to create a proposal and manually mark it accepted to generate an invoice. (If this is too large, it might be partially implemented or pushed to next phase; however, having at least one end-to-end example of external workflow could be part of MVP).
- **User management & roles** setup, so that we can have at least admin, one manager, etc., in the system using Firebase Auth.
- **Notifications via email** for key events (submission, approval, etc.), and basic reminder before contract expiry as an email.
- The application running on web (desktop/mobile web) with React, connected to Firebase backend.

- **Future Enhancements (Post-MVP):**

- Support for **additional workflow types** (e.g. Purchase Orders if not in MVP, or others like *travel request approvals*, *IT access requests*, etc.) with an interface for admins to add new templates.
- More advanced **contract management features**: vendor performance metrics, cost analysis of renewals, and compliance tracking as seen in dedicated tools ⁶ ⁷. For example, capturing

usage data of software to decide if renewal is necessary, or generating reports of renewal costs over time.

- **Integration with external systems:** e.g. connecting to an ERP for automatically creating a purchase order in that system once approved, or integrating with calendar for contract dates, Slack/Teams for notifications, etc. Also, integration of online **payment gateways** for invoices (Stripe/PayPal) to facilitate the payment link feature.
- **Enhanced analytics and reporting:** a reports module where admins can run custom reports on all data (requests by type, approval times, expenditures, etc.), and perhaps export to CSV. Also, adding charts/graphs in the dashboard (if not already done) and maybe a KPI view for management.
- **Mobile application:** Develop the React Native app for iOS/Android to allow push notifications and on-the-go approvals. This will increase adoption for users who prefer mobile.
- **Customization & Branding:** Allow companies to configure the look (logo, colors) of the app and tweak workflows on their own. Also, possibly support **multi-tenant** setup where multiple companies use the service in isolation (if offering as SaaS to many clients).
- **Template Library:** Expand the proposal template system, and perhaps include templates for common contracts or documents. Also, saving frequently used line items or clauses for quick reuse.
- **Audit and Compliance Features:** e.g. ability to extract an audit trail report, set retention policies for data, ensure the system meets standards (like GDPR, SOC2 if needed when scaling up as a SaaS product).
- **Performance and Scalability:** As usage grows, optimize the Firestore data structures and Cloud Function performance. (Firebase should handle quite a lot by default, but monitoring and scaling for large datasets will be considered.)

Scrum Implementation Note: All the above features will be refined into detailed user stories and tasks during sprint planning. The team will work in iterative sprints (e.g., 2-week cycles) delivering incremental improvements. Early sprints will focus on the core architecture (Firebase setup, basic React app structure, auth) and then implementing the highest-value features (likely the expense approval and contract reminder features first). Regular demos will be conducted to gather feedback. The backlog above is a living artifact – we will reprioritize and adjust details as we learn more from actual usage and user input.

Overall, this finance project aims to deliver a cohesive solution that **brings efficiency, visibility, and control** to various financial workflows in a business. By organizing the development with Agile/Scrum, we ensure that we can adapt to changes quickly and deliver a product that closely aligns with user needs. Each feature set (from simple approval flows to integrated invoicing) will be built, tested, and improved iteratively, driving the project from a basic MVP to a robust platform over time.

1 2 3 6 7 Top 10 Contract Renewal Management Software [2025 Updated] | Zluri
<https://www.zluri.com/blog/contract-renewal-management-software>

4 5 Seamless Integration: Best Practices for Combining Invoicing and Proposal
<https://invoicecrowd.com/seamless-integration-best-practices-for-combining-invoicing-and-proposal-systems/>